

Queue ADT Using Arrays

FIFO Principle, Array Representation and C Implementation

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

What is a Queue?

Definition

A queue is a linear data structure in which insertion is performed at one end, called the **rear**, and deletion is performed at the other end, called the **front**.

A queue follows:

FIFO

First In, First Out

What is a Queue?

Definition

A queue is a linear data structure in which insertion is performed at one end, called the **rear**, and deletion is performed at the other end, called the **front**.

A queue follows:

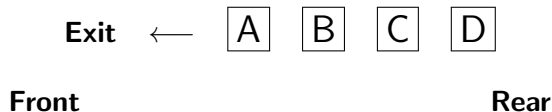
FIFO

First In, First Out

The element inserted first is removed first.

Real-Life Analogy

Consider people waiting at a service counter.



- New elements enter from the rear.
- Existing elements leave from the front.
- The first inserted element is deleted first.

Queue Order Example

Suppose the following values are inserted:

10, 20, 30, 40

Insertion order:

10 → 20 → 30 → 40

Deletion order:

10 → 20 → 30 → 40

Queue Order Example

Suppose the following values are inserted:

10, 20, 30, 40

Insertion order:

10 → 20 → 30 → 40

Deletion order:

10 → 20 → 30 → 40

Observation

Unlike a stack, the insertion order and deletion order are the same.

Stack vs Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion	Top	Rear
Deletion	Top	Front
Main variables	Top	Front and rear
Example	Plate stack	Waiting line

Queue as an Abstract Data Type

A Queue ADT defines the permitted operations without fixing the internal implementation.

Typical operations are:

- `create()`
- `enqueue(value)`
- `dequeue()`
- `peek()`
- `isEmpty()`
- `isFull()`
- `size()`
- `display()`

Queue as an Abstract Data Type

A Queue ADT defines the permitted operations without fixing the internal implementation.

Typical operations are:

- `create()`
- `enqueue(value)`
- `dequeue()`
- `peek()`
- `isEmpty()`
- `isFull()`
- `size()`
- `display()`

A queue may be implemented using:

- Arrays
- Linked lists
- Circular arrays

Queue Terminology

Term	Meaning
Enqueue	Insert an element at the rear
Dequeue	Remove an element from the front
Front	Position of the first active element
Rear	Position of the last active element
Overflow	Insertion attempted when full
Underflow	Deletion attempted when empty
Peek	Read the front element without deletion

Array Representation of a Queue

Assume:

$$MAX = 10, \quad front = 0, \quad rear = 5$$

12	9	7	18	14	36				
----	---	---	----	----	----	--	--	--	--

0 1 2 3 4 5 6 7 8 9

```

      ↑                               ↑
front = 0      rear = 5

```

Role of Front and Rear

- front points to the first active element.
- rear points to the last active element.
- Enqueue changes rear.
- Dequeue changes front.

If the queue contains:

12, 9, 7, 18, 14, 36

then:

`front = 0`

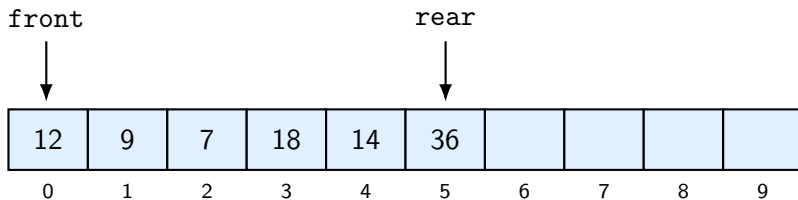
`rear = 5`

Visual Representation

Assume:

$$MAX = 10$$

$$front = 0, \quad rear = 5$$



Queue Invariants

For a valid non-empty linear queue:

$$0 \leq \text{front} \leq \text{rear} < \text{MAX}$$

Active queue elements occupy:

`queue[front]`

through:

`queue[rear]`

The number of active elements is:

`rear-front+1`

For an empty queue:

`front=rear=-1`

Empty and Full Conditions

Queue is Empty

```
front == -1
```

or:

```
front > rear
```

Linear Queue is Full

```
rear == MAX-1
```

In a clean implementation, the queue is reset to:

```
front=rear=-1
```

after the last element is removed.

Enqueue Operation

Enqueue inserts a new value at the rear.

Suppose:

$\text{front} = 0, \quad \text{rear} = 2$

and:

10, 20, 30

Enqueue 40:

- 1 Check for overflow.
- 2 Increment rear.
- 3 Store the new value at `queue[rear]`.

Result:

10, 20, 30, 40

First Enqueue Operation

Initially:

```
front=-1
```

```
rear=-1
```

When the first value is inserted:

```
front=0
```

```
rear=0
```

Then:

```
queue[rear]=value
```

First Enqueue Operation

Initially:

```
front=-1
```

```
rear=-1
```

When the first value is inserted:

```
front=0
```

```
rear=0
```

Then:

```
queue[rear]=value
```

Both variables point to the only element in the queue.

Enqueue Algorithm

Algorithm: ENQUEUE

① If `rear == MAX-1`, report overflow.

② If the queue is empty:

`front = rear = 0`

③ Otherwise:

`rear = rear + 1`

④ Store:

`queue[rear] = value`

Enqueue complexity = $O(1)$

Queue Overflow

Overflow occurs when:

$$\text{rear} == \text{MAX}-1$$

Example:

$$\text{MAX} = 5$$

$$\text{rear} = 4$$

All positions from index 0 through 4 have already been reached.

Any further enqueue causes:

QUEUE OVERFLOW

Enqueue Function in C

```
1 int enqueue(int value)
2 {
3     if (rear == MAX - 1)
4     {
5         printf("Queue overflow\n");
6         return 0;
7     }
8
9     if (front == -1)
10    {
11        front = 0;
12    }
13
14    rear++;
15    queue[rear] = value;
16
17    return 1;
18 }
```

Return Value

The function returns 1 for success and 0 for failure

Deque Operation

Deque removes the element from the front.

Suppose:

$\text{front} = 0, \quad \text{rear} = 3$

and:

10, 20, 30, 40

Deque removes:

10

After deletion:

$\text{front} = 1$

The active queue becomes:

Deque Algorithm

Algorithm: DEQUEUE

① If the queue is empty, report underflow.

② Store:

`value=queue[front]`

③ If the queue contains only one element:

`front=rear=-1`

④ Otherwise:

`front=front+1`

⑤ Return the removed value.

Deque complexity = $O(1)$

Removing the Last Element

Suppose:

`front=rear=3`

The queue contains only:

50

After dequeue, no active element remains.

Therefore, reset:

`front=rear=-1`

Removing the Last Element

Suppose:

$$\text{front}=\text{rear}=3$$

The queue contains only:

50

After dequeue, no active element remains.

Therefore, reset:

$$\text{front}=\text{rear}=-1$$

This makes the empty-state representation consistent.

Queue Underflow

Underflow occurs when dequeue or peek is attempted on an empty queue.

Empty condition:

```
front == -1
```

There is no valid value at:

```
queue[front]
```

Therefore:

```
QUEUE UNDERFLOW
```

Deque Function in C

```
1 int dequeue(int *removedValue)
2 {
3     if (front == -1)
4     {
5         printf("Queue underflow\n");
6         return 0;
7     }
8
9     *removedValue = queue[front];
10
11     if (front == rear)
12     {
13         front = -1;
14         rear = -1;
15     }
16     else
17     {
18         front++;
19     }
20
21     return 1;
22 }
```

Peek Operation

Peek returns the element at the front without deleting it.

Suppose:

`front = 2`

and the active queue is:

30, 40, 50

Then:

`peek() = 30`

After peek:

`front = 2`

`rear`

Peek Function in C

```
1 int peek(int *frontValue)
2 {
3     if (front == -1)
4     {
5         printf("Queue is empty\n");
6         return 0;
7     }
8
9     *frontValue = queue[front];
10    return 1;
11 }
```

Peek complexity = $O(1)$

Display Operation

To display the active queue:

```
for(i=front; i<=rear; i++)
```

Example:

```
front = 2
```

```
rear = 5
```

Active elements:

```
queue[2], queue[3], queue[4], queue[5]
```

Time complexity:

$O(n)$

Display Function in C

```
1 void display(void)
2 {
3     int i;
4
5     if (front == -1)
6     {
7         printf("Queue is empty\n");
8         return;
9     }
10
11     printf("FRONT -> ");
12
13     for (i = front; i <= rear; i++)
14     {
15         printf("%d ", queue[i]);
16     }
17
18     printf("<- REAR\n");
19 }
```

Queue Dry Run

Assume:

$$MAX = 5$$

Initially:

`front=rear=-1`

Perform:

ENQUEUE(10)

ENQUEUE(20)

ENQUEUE(30)

DEQUEUE()

Queue Dry Run

Assume:

$$MAX = 5$$

Initially:

`front=rear=-1`

Perform:

ENQUEUE(10)

ENQUEUE(20)

ENQUEUE(30)

DEQUEUE()

Detailed Operation Table

Step	Operation	Front	Rear	Active Queue
0	Initial	-1	-1	Empty
1	Enqueue 10	0	0	10
2	Enqueue 20	0	1	10, 20
3	Enqueue 30	0	2	10, 20, 30
4	Dequeue	1	2	20, 30
5	Enqueue 40	1	3	20, 30, 40

Interactive Trace

Assume:

$$MAX = 4$$

Perform:

ENQUEUE(5)

ENQUEUE(8)

DEQUEUE()

ENQUEUE(12)

ENQUEUE(20)

Interactive Trace

Assume:

$$MAX = 4$$

Perform:

ENQUEUE(5)

ENQUEUE(8)

DEQUEUE()

ENQUEUE(12)

ENQUEUE(20)

Complexity of Queue Operations

Operation	Time Complexity	Reason
Enqueue	$O(1)$	Direct insertion at rear
Dequeue	$O(1)$	Direct removal at front
Peek	$O(1)$	Direct access at front
isEmpty	$O(1)$	One condition check
isFull	$O(1)$	One condition check
Size	$O(1)$	Arithmetic using indices
Display	$O(n)$	Visits all active elements

Space complexity:

$$O(MAX)$$

Why Does Dequeue Not Shift Elements?

A poor implementation may shift every element left after deletion.
That would make dequeue:

$$O(n)$$

Instead, we only increment:

`front`

Therefore:

$$\text{Dequeue} = O(1)$$

Why Does Dequeue Not Shift Elements?

A poor implementation may shift every element left after deletion.
That would make dequeue:

$$O(n)$$

Instead, we only increment:

`front`

Therefore:

$$\text{Dequeue} = O(1)$$

The logically deleted positions remain unused in a linear queue.

Major Limitation: False Overflow

Assume:

$$MAX = 5$$

After several operations:

$$front = 2$$

$$rear = 4$$

The queue may contain only three elements, but:

$$rear = MAX - 1$$

Therefore, another enqueue reports overflow.

Major Limitation: False Overflow

Assume:

$$MAX = 5$$

After several operations:

$$front = 2$$

$$rear = 4$$

The queue may contain only three elements, but:

$$rear = MAX - 1$$

Therefore, another enqueue reports overflow.

Positions 0 and 1 are empty but cannot be reused.

This situation is called:

False Overflow in a Linear Queue

Assume:

$MAX = 5$, $front = 2$, $rear = 4$

Deleted	Deleted	30	40	50
0	1	2	3	4

$front = 2$ $rear = 4$

↑ ↑

Problem

Although positions 0 and 1 are empty, another element cannot be inserted because $rear = MAX - 1$.

Solution to False Overflow

Possible solutions include:

- Shift active elements towards the beginning.
- Use a circular queue.
- Use a linked-list implementation.

Solution to False Overflow

Possible solutions include:

- Shift active elements towards the beginning.
- Use a circular queue.
- Use a linked-list implementation.

Shifting requires:

$$O(n)$$

time.

A circular queue reuses empty locations efficiently while maintaining:

$$O(1)$$

enqueue and dequeue operations.

Definitions and Helper Functions

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 int isEmpty(void)
10 {
11     return front == -1;
12 }
13
14 int isFull(void)
15 {
16     return rear == MAX - 1;
17 }
18
19 int size(void)
20 {
21     if (isEmpty())
22         return 0;
```

Enqueue Implementation

```
1 int enqueue(int value)
2 {
3     if (isFull())
4     {
5         printf(
6             "Queue overflow: cannot insert %d\n",
7             value
8         );
9
10        return 0;
11    }
12
13    if (isEmpty())
14    {
15        front = 0;
16    }
17
18    rear++;
19    queue[rear] = value;
20
21    printf("%d inserted successfully\n", value);
22    return 1;
```

Deque Implementation

```
1 int dequeue(int *removedValue)
2 {
3     if (isEmpty())
4     {
5         printf("Queue underflow\n");
6         return 0;
7     }
8
9     *removedValue = queue[front];
10
11     if (front == rear)
12     {
13         front = -1;
14         rear = -1;
15     }
16     else
17     {
18         front++;
19     }
20
21     return 1;
22 }
```

Peek and Display

```
1 int peek(int *frontValue)
2 {
3     if (isEmpty())
4     {
5         printf("Queue is empty\n");
6         return 0;
7     }
8
9     *frontValue = queue[front];
10    return 1;
11 }
12
13 void display(void)
14 {
15     int i;
16
17     if (isEmpty())
18     {
19         printf("Queue is empty\n");
20         return;
21     }
22 }
```

Main Function: Menu

```
1 int main(void)
2 {
3     int choice;
4     int value;
5
6     do
7     {
8         printf("\n--- LINEAR QUEUE MENU ---\n");
9         printf("1. Enqueue\n");
10        printf("2. Dequeue\n");
11        printf("3. Peek\n");
12        printf("4. Display\n");
13        printf("5. Size\n");
14        printf("6. Check Empty\n");
15        printf("7. Check Full\n");
16        printf("8. Exit\n");
17        printf("Enter choice: ");
18
19        if (scanf("%d", &choice) != 1)
20        {
21            printf("Invalid input\n");
22            return 1;
23        }
24    }
```

Main Function: Operations

```
1  switch (choice)
2  {
3  case 1:
4      printf("Enter value: ");
5
6      if (scanf("%d", &value) != 1)
7      {
8          printf("Invalid value\n");
9          return 1;
10     }
11
12     enqueue(value);
13     break;
14
15 case 2:
16     if (dequeue(&value))
17     {
18         printf(
19             "Deleted value: %d\n",
20             value
21         );
22     }
```

Main Function: Remaining Cases

```
1  case 4:
2      display();
3      break;
4
5  case 5:
6      printf(
7          "Current queue size: %d\n",
8          size()
9      );
10     break;
11
12 case 6:
13     printf(
14         isEmpty()
15         ? "Queue is empty\n"
16         : "Queue is not empty\n"
17     );
18     break;
19
20 case 7:
21     printf(
22         isFull()
```

Complete Program: Part 1

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 int isEmpty(void)
10 {
11     return front == -1;
12 }
13
14 int isFull(void)
15 {
16     return rear == MAX - 1;
17 }
18
19 int size(void)
20 {
21     if (isEmpty())
22         return 0;
```

Complete Program: Part 2

```
1 int dequeue(int *removedValue)
2 {
3     if (isEmpty())
4     {
5         printf("Queue underflow\n");
6         return 0;
7     }
8
9     *removedValue = queue[front];
10
11     if (front == rear)
12     {
13         front = -1;
14         rear = -1;
15     }
16     else
17     {
18         front++;
19     }
20
21     return 1;
22 }
```

Complete Program: Part 3

```
1 void display(void)
2 {
3     int i;
4
5     if (isEmpty())
6     {
7         printf("Queue is empty\n");
8         return;
9     }
10
11     printf("FRONT -> ");
12
13     for (i = front; i <= rear; i++)
14         printf("%d ", queue[i]);
15
16     printf("<- REAR\n");
17 }
18
19 int main(void)
20 {
21     int choice;
22     int value;
```

Complete Program: Part 4

```
1  switch (choice)
2  {
3  case 1:
4      printf("Enter value: ");
5      scanf("%d", &value);
6      enqueue(value);
7      break;
8
9  case 2:
10     if (dequeue(&value))
11         printf("Deleted: %d\n", value);
12     break;
13
14 case 3:
15     if (peek(&value))
16         printf("Front: %d\n", value);
17     break;
18
19 case 4:
20     display();
21     break;
```

Applications of Queues

Queues are commonly used in:

- CPU process scheduling
- Printer scheduling
- Keyboard input buffering
- Network packet processing
- Customer-service systems
- Breadth-first search
- Web-server request handling
- Message queues
- Call-centre systems
- Simulation of waiting lines

CPU Scheduling Example

Processes arrive in this order:

P_1, P_2, P_3, P_4

In First-Come, First-Served scheduling:

P_1

is executed first, followed by:

P_2, P_3, P_4

This is FIFO behaviour.

Queue

Printer Queue Example

Print jobs are submitted as:

Job_1, Job_2, Job_3

The printer generally processes:

Job_1

before:

Job_2

and:

Job_3

The first submitted job is normally printed first.

Rapid-Fire Questions

- 1 What does FIFO mean?

Rapid-Fire Questions

- 1 What does FIFO mean?
- 2 At which end is insertion performed?

Rapid-Fire Questions

- 1 What does FIFO mean?
- 2 At which end is insertion performed?
- 3 At which end is deletion performed?

Rapid-Fire Questions

- ① What does FIFO mean?
- ② At which end is insertion performed?
- ③ At which end is deletion performed?
- ④ What is the initial value of front and rear?

Rapid-Fire Questions

- ① What does FIFO mean?
- ② At which end is insertion performed?
- ③ At which end is deletion performed?
- ④ What is the initial value of front and rear?
- ⑤ When does linear queue overflow occur?

Rapid-Fire Questions

- 1 What does FIFO mean?
- 2 At which end is insertion performed?
- 3 At which end is deletion performed?
- 4 What is the initial value of front and rear?
- 5 When does linear queue overflow occur?
- 6 What is false overflow?

Multiple-Choice Question

A queue has:

$\text{front} = 2$

$\text{rear} = 6$

How many active elements are present?

- ① 4
- ② 5
- ③ 6
- ④ 7

Multiple-Choice Question

A queue has:

$$\text{front} = 2$$

$$\text{rear} = 6$$

How many active elements are present?

- ① 4
- ② 5
- ③ 6
- ④ 7

$$\text{Size} = \text{rear} - \text{front} + 1$$

$$= 6 - 2 + 1$$

Find the Error: Enqueue

```
1 void enqueue(int value)
2 {
3     rear++;
4     queue[rear] = value;
5
6     if (rear == MAX - 1)
7         printf("Overflow");
8 }
```

What is wrong?

Find the Error: Enqueue

```
1 void enqueue(int value)
2 {
3     rear++;
4     queue[rear] = value;
5
6     if (rear == MAX - 1)
7         printf("Overflow");
8 }
```

What is wrong?

Overflow must be checked before incrementing rear and storing the value.

Otherwise, the program may write outside the array.

Find the Error: Dequeue

```
1 int dequeue(void)
2 {
3     front++;
4     return queue[front];
5 }
```

Problems:

- No underflow check
- Front is incremented before reading the value
- The wrong element is returned
- Empty-state reset is missing

Correct order:

`value=queue[front]`

followed by updating front.

Classroom Exercise

Assume:

$$MAX = 5$$

Perform:

ENQUEUE(15)

ENQUEUE(25)

ENQUEUE(35)

DEQUEUE()

ENQUEUE(45)

Classroom Exercise: Answer

After three enqueue operations:

15, 25, 35

Dequeue removes:

15

After enqueueing 45:

25, 35, 45

Peek returns:

25

Final indices:

front = 1

Answer the following:

- 1 Differentiate between a stack and a queue.
- 2 Why are two variables required for a queue?
- 3 What is the queue overflow condition?
- 4 Why is resetting front and rear necessary after deleting the last element?
- 5 Explain false overflow with an example.

References

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, Computer Science Press.
- ② Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.
- ③ Reema Thareja, *Data Structures Using C*, Oxford University Press.

Questions?

Thank You